

目 录

一、引言

- 1.1 编写目的
- 1.2 软件概述
- 1.3 主要特点
- 1.4 运行环境
- 1.5 术语与缩略语

二、游戏全流程说明

- 2.1 启动与健康游戏忠告
- 2.2 厨房主界面
- 2.3 冰箱与烹饪
- 2.4 厨艺升级与内容解锁
- 2.5 家具升级与图鉴收集
- 2.6 出门选点与交通工具
- 2.7 集市卡片路线玩法
- 2.8 摊面翻堆与食材检视
- 2.9 菜篮装箱与干湿分区
- 2.10 集市事件与油纸菜谱
- 2.11 收摊结算与带货回家
- 2.12 特殊市场节奏玩法

三、游戏元素说明

- 3.1 食材
- 3.2 摊型与路线卡片
- 3.3 菜谱与菜品
- 3.4 资源与成长

四、可进入场景一览

五、游戏操作方法

- 5.1 基本操作
- 5.2 天色与撤离规则
- 5.3 菜篮放置规则
- 5.4 经济与成长规则
- 5.5 每日限量规则

六、软件总体设计

- 6.1 软件需求概括
- 6.2 总体架构设计
- 6.3 模块划分与关系

6.4 场景与浮层系统设计

6.5 主循环与资源加载设计

七、核心模块详细设计

7.1 游戏入口与平台适配模块

7.2 场景管理与厨房主场景模块

7.3 集市地图与卡片路线模块

7.4 翻堆抽货与品质模块

7.5 菜篮装箱模块

7.6 结算与冰箱模块

7.7 烹饪与成长模块

7.8 特殊市场模块

7.9 图鉴模块

7.10 存档与云同步模块

7.11 音频与 CDN 资源模块

八、数据结构设计

九、数据接口设计

十、出错处理设计

十一、性能优化设计

十二、结论

一、引言

1.1 编写目的

本文档是深圳幸运呱科技有限公司扫荡菜场小游戏软件 V1.0.0 软件著作权登记文档鉴别材料。文档说明本软件的功能范围、游戏全流程、可进入场景、操作方法、总体架构、核心模块、数据结构、接口设计、异常处理与性能优化方案，用以证明本软件为独立开发完成的原创游戏软件。

本文档面向软件著作权审查人员及后续维护人员，重点描述软件技术实现与玩法规则，不包含运营数据、用户隐私数据与商业敏感策略。

1.2 软件概述

扫荡菜场是一款运行于微信小游戏平台的休闲经营类小游戏软件。玩家扮演一位过日子的居家玩家，在菜场收摊时段付钱买下摊主剩下的整堆货，在老板装箱收摊之前把堆翻开、逐件检视品质，并塞进容量有限的出门菜篮，赶在天黑前撤离；回家后把食材放进冰箱，卖掉换钱，或按菜谱下锅做菜，用做菜获得的厨艺经验提升等级，进而解锁更远的菜场、更多菜谱与更大的家具。

本软件的主要功能包括：

- * 厨房主场景：以自家厨房为中枢，通过点击冰箱、烹饪台、菜篮、泡沫箱、门等热区进入各功能面板。
- * 出门选点系统：按厨艺等级与已购交通工具开放九个常规菜场与三个特殊市场，出门消耗体力。
- * 集市卡片路线玩法：一局按“天色”步数计算，玩家沿多排卡片路线逐张挑选前进，走完天黑收摊。
- * 翻堆与检视系统：付费买下摊位剩货后翻开菜堆随机抽出食材，逐件检视得到真实品质与售价。
- * 菜篮装箱系统：出门篮按占格网格管理，分左湿区、右干区与广告解锁通用行，食材按尺寸拖入并可旋转。
- * 收摊结算系统：区分从容收工与天黑被赶两种结算，可勾选当场卖出或带回冰箱。
- * 冰箱与烹饪系统：冰箱按格位与叠放份数管理食材与熟菜；烹饪台按菜谱校验食材并下锅出菜。
- * 成长与解锁系统：厨艺 1-15 级，家具与房屋分级升级，菜谱通过烹饪台、厨艺等级与集市油纸三条线解锁。
- * 图鉴收集系统：按食材分类与菜品分组记录已见与已检视条目，展示收录进度。
- * 存档与云同步系统：本地存储与腾讯云 CloudBase HTTP 后端双写，按平台隔离账号并做版本冲突校验。

1.3 主要特点

- * 玩法原创：以“收摊捡漏”为核心情境，把有限容量的背包装箱玩法与菜场路线选择结合，不使用市面常见的抽卡或三消结构。
- * 占格装箱：每种食材按真实体积定义占格宽高（例如带鱼 1x4、冬瓜 3x2），装箱需要考虑旋转、干湿分区与易碎品相邻规则。

- * 一次性局内随机：每局用整数种子驱动伪随机数生成器构建卡片路线与摊位货堆，同一种子可复现，便于测试与数值校验。
- * 玩法逻辑与渲染分离：src/sim 目录为纯函数模拟层，不依赖渲染对象，可独立运行数值审计脚本。
- * 跨平台适配层：通过 PlatformService 统一封装微信、抖音与 H5 的请求、存储、登录与生命周期接口。
- * 资源分包与 CDN：主包只保留启动必需资源，图片与音频按分包与云端 CDN 分发并做本地文件缓存。

1.4 运行环境

环境项	要求说明
客户端平台	微信小游戏；平台适配层已预留抖音小游戏与 H5 浏览器
开发语言	TypeScript、JavaScript
渲染技术	PixiJS 7 + WebGL，不支持时逐级降级到 Canvas
设计分辨率	750 x 1334 竖屏，按屏幕宽度等比缩放并适配安全区
构建工具	Vite 6 + TypeScript 5，打包为单文件 IIFE bundle
后端平台	腾讯云 CloudBase 云函数 + HTTP 访问服务
数据存储	微信 Storage / 浏览器 localStorage / CloudBase 文档数据库

1.5 术语与缩略语

术语	含义说明
天色	一局的步数预算。走一张卡消耗一步，走完即天黑收摊
翻堆	买下摊位剩货后翻开菜堆，随机抽出其中食材的操作
检视	对已抽出食材做一次品质判定，检视后才显示真名与真实售价
出门篮	出门携带的容器组合：左侧泡沫箱管湿区，右侧塑料袋管干区
占格	食材在菜篮网格中占据的宽高格数，由 ItemDef 的 w、h 字段定义
神检	每日限量一次的稀有食材揭示事件，把普通小鱼替换为野生大黄鱼
油纸	集市卡片上可检到的菜谱纸，是菜谱的第三条解锁来源
厨艺	玩家等级（1-15 级），仅通过做菜获得经验，决定菜场与家具解锁
GAME_KEY	游戏代号 jiancai，用于存档 key、Token key 与后端集合名的多游戏隔离
JWT	JSON Web Token，后端鉴权与平台账号识别凭证

二、游戏全流程说明

本章按玩家实际操作顺序，图文结合说明本软件从启动到完整循环的全部流程。软件不设独立开始页，加载完成后直接进入厨房主界面。

2.1 启动与健康游戏忠告

玩家进入小游戏后首先显示加载界面。界面展示加载底图、游戏名称 Logo、一条以鱼形指示物推进的加载进度条，以及底部完整的《健康游戏忠告》文本。

加载阶段并行完成四件事：云同步预热与启动拉取（超时 2500 毫秒即放行）、本地存档读取与体力恢复结算、CDN 资源清单获取、首屏纹理与音效预加载。全部完成或达到 12000 毫秒上限后，至少停留 480 毫秒再切入厨房主界面，避免闪屏。



图1 启动加载界面与健康游戏忠告

2.2 厨房主界面

厨房是全部流程的中枢。场景为一张可横向平移的宽幅厨房图，随房屋等级变化布局。顶部常驻信息条显示金币、体力（当前值/上限 5）与厨艺等级，右上角为静音开关，左侧竖排为图鉴入口。

厨房内可点击的热区及其去向如下：

热区	点击后进入	说明
门	出门选点界面	选择菜场、交通工具与特殊市场，开始一局收摊
冰箱	冰箱面板	查看、卖出或吃掉冰箱中的食材与熟菜

烹饪台	烹饪面板	选择已解锁菜谱，校验食材并下锅做菜
菜篮	升级面板	升级出门塑料袋/菜篮，扩大干区格数
泡沫箱	升级面板	升级出门泡沫箱/水桶，扩大湿区格数
家具上方箭头	升级浮层	按金币与厨艺门槛升级冰箱、烹饪台等家具
图鉴图标	图鉴面板	查看食材与菜品收录进度



图2 厨房主界面

2.3 冰箱与烹饪

一局结束带回的食材统一进入冰箱，冰箱内部不再区分干湿。冰箱按格位管理，同种食材或同种熟菜可在同一格内叠放，单格上限 10 份；总格数随冰箱等级从 18 格提升到 72 格，每升一级增加 6 格。冰箱面板可查看每件条目的名称、份数与单价，支持卖出换取金币，熟菜还可直接吃掉以恢复体力。



图3 冰箱面板

烹饪面板列出当前已解锁的菜谱。每道菜谱声明所需食材类别与份数，面板实时对照冰箱库存标出缺口；材料齐备时可点击下锅，扣除对应食材并产出一份带售价的熟菜存入冰箱，同时结算厨艺经验。菜谱首次做成时给出更高的一次性经验奖励。



图4 烹饪面板 - 选菜谱下锅

2.4 厨艺升级与内容解锁

厨艺等级是唯一的主线成长轴，只能通过做菜积累经验，共 15 级。经验来源由菜谱稀有度与出菜份数共同决定：稀有度基础经验为普通 3、稀有 8、史诗 16、传说 28，另按份数追加，并乘以对应稀有度的烹饪系数。升级所需经验按等级递增，从 60 点起逐级抬升至 1420 点。

升级时弹出厨艺升级窗口，集中展示本级新开放的内容：新的菜场、新解锁的菜谱，以及可继续升级的家具门槛。菜场按厨艺 1/3/5/7/9/11/12/13/14 逐个开放。



图5 厨艺升级弹窗

2.5 家具升级与图鉴收集

厨房共四件可升级家具：冰箱、烹饪台、菜篮（出门塑料袋）与泡沫箱（出门湿区容器），各自 0-9 级。升级费用按等级指数递增，公式为基准价乘以 1.38 的等级次方；大件家具还受房屋等级限制。房屋分陋屋、精装屋、雅致屋三档，升级分别需要厨艺 4 级与 8 级，费用为 380 与 980 金币。



图6 家具升级面板

家具与房屋升级不只改数值，还会立刻替换厨房里的展示图。冰箱、烹饪台、菜篮、泡沫箱各自按等级更换贴图；房屋从陋屋升到精装屋、雅致屋时，整房底图、墙面、地面与家具摆位一并换装。玩家从画面就能辨认当前进度。图2 为开局陋屋外观，图7 为多次升级后的厨房，两者对照可见展示图随等级变化。



图7 家具升级后的厨房展示

图鉴面板分食材与菜品两个大页。食材页按叶菜、根茎、蛋豆、水产、肉禽等分类展示，区分“见过”与“已检视”两种收录状态；菜品页按家常、凉菜、汤羹、水产、荤菜等分组展示已做成的菜。未收录条目以灰度剪影显示，只给出占格轮廓，不泄露名称。



图8 图鉴面板

2.6 出门选点与交通工具

点击厨房的门进入出门选点界面。界面顶部显示厨艺等级、体力、冰箱剩余空位，中部是可竖向滚动的菜场卡片列表，每张卡给出菜场名称、一句风味提示、缩略图与解锁条件；底部是交通工具停放区，可左右切换浏览。

交通工具共四档：走路（开局拥有）、自行车 180 金币、电动车 420 金币、小货车 860 金币，必须按顺序购买，跳级显示为剪影。每档交通工具决定“看得见哪些菜场”，能否真正进入还要看厨艺等级是否达标。每档交通工具还对应一个特殊市场入口。出门消耗 1 点体力，体力上限 5 点，每 30 分钟自然恢复 1 点。



图9 出门选点界面



图10 交通工具购买

2.7 集市卡片路线玩法

选定菜场后播放出门过渡动画并进入集市场景。一局的推进单位是“天色”步数：路线以多排卡片呈现，每排提供二至三张候选卡，玩家点选其中一张前进一步并消耗一点天色，天色走完即天黑收摊。

各菜场的步数预算为 10-15 步，摊位层数 3-5 层。

顶部信息条常驻显示剩余天色、金币、冰箱剩余空位、菜篮干湿两区已占计数，以及回家按钮。走过的卡型会被记忆，下次在同一菜场再遇到相同卡型时以明牌显示，明牌给出摊型图标与“免费/需付 N 金币，剩 N 件”的提示；未见过的卡型以背面暗牌显示。



图11 集市卡片路线

2.8 摊面翻堆与食材检视

选中摊位卡后进入翻堆界面。收摊剩货需要先付进场费买下（各菜场 1-5 金币，水产摊与肉摊另加 2-3 金币），免费卡则不需付费但坏货概率明显更高：付费买下的堆坏货概率 6%，免费捡的堆坏货概率 18%。

界面上是若干遮挡菜堆，点击即随机抽出一件食材放到摊桌上。刚抽出的食材只显示模糊类别名，需要点击进入检视卡才会揭示真名、品质、新鲜度、占格尺寸与预计售价。品质分坏、普通、新鲜、上品与神捡五档。



图12 摊面翻堆界面



图13 食材检视卡

摊主装箱是本局的时间压力来源：停留在同一摊位时装箱进度按摊型速率持续推进，进度满 100 即被清场，未装进菜篮的桌上食材作废。

2.9 菜篮装箱与干湿分区

菜篮面板是本作的核心操作界面。出门篮由两个容器拼成：左侧泡沫箱构成湿区，右侧塑料袋构成干区，最上方预留一整行广告解锁的通用行，解锁后干湿皆可放且只在本局有效。干区列数随菜篮等级由 4 列增到 8 列，湿区列数随泡沫箱等级由 2 列增到 6 列，行数同步成长。菜篮等级达到 3 级后获得保温底，干区最底一行可当湿格使用。

每件食材按 ItemDef 中的宽高占格放置，可整体旋转 90 度。放置合法性校验包含：不越界、不与已有食材重叠、湿货只能进湿格或通用行、活物必须贴底放、蛋类等易碎品与豆腐等易压品有相邻限制。放置失败时可尝试挤走已有食材后自动重排。



图14 菜篮装箱面板

2.10 集市事件与油纸菜谱

路线卡片除摊位外还有多种事件卡：白捡（直接获得一件食材）、空摊、死路（浪费一步）、人情（摊主赠送）、对话、采集小游戏、分岔与关卡门等。对话卡弹出事件面板，展示摊主或熟客半身像、台词与若干选项，选择后产生不同结果。



图15 集市事件对话

油纸菜谱是菜谱的第三条解锁来源，不进入公共卡片权重池：每局单独掷一次概率决定是否放置一张，各菜场概率为 0.36-0.70；在某个菜场从未捡到过油纸时，本局保底放置一张。捡到后弹出菜谱解锁窗口展示菜名、所需食材与稀有度。



图16 油纸菜谱卡片



图17 油纸菜谱解锁弹窗

“神捡”为每日限量一次的稀有事件：满足条件时以 85% 概率把水产摊中的一条普通小鱼替换为野生大黄鱼，抽出时播放专属揭示特效，检视后才显示真名与专属售价。

2.11 收摊结算与带货回家

结算分两种。玩家在天色用尽前主动点击回家为“从容收工”，菜篮内全部食材完整带回；天色走完被赶出集市为“天黑收摊”，可能损失部分新鲜度或掉落物品。

结算面板逐件列出带回的食材、品质与售价，玩家可以勾选当场卖出换金币，未勾选的进入冰箱。当冰箱剩余空位不足时，面板强制要求先勾选卖出若干件腾出空间，确认后才完成入库。



图18 从容收工结算



图19 天黑收摊结算

2.12 特殊市场节奏玩法

特殊市场是与卡片路线并列的第二种局内玩法，共三个：香料夜摊（走路）、江边垂钓（自行车）、老城干货（电动车），各自每日限进 2 次。玩法为节奏点击：每局 5 轮，每轮依次经过静默、预告、

命中窗口、过迟与结算五个阶段，玩家需在命中窗口（0.4 秒）内点击，命中获得目标货池食材，过早或过迟则落入安慰货池。五轮结束后进入与常规收摊一致的结算面板。



图20 特殊市场节奏玩法

三、游戏元素说明

3.1 食材

软件共定义 77 种常规食材，另有 1 种神捡专属食材野生大黄鱼，合计 78 种。每种食材由 ItemDef 描述，字段包含唯一 id、中文名、占格宽高 w/h、干湿分区 zone、稀有度 rarity、可出现的摊型 stalls、主色、以及易碎、活物、大件、坚硬、易压等物理标记与风味描述文本。

售价由占格面积与稀有度共同推导，具体规则如下：

计价项	取值	说明
每格单价	普通 1.5 / 稀有 3 / 史诗 5 / 传说 8	按稀有度取值，乘以占格面积
稀有度保底	普通 0.5 / 稀有 3 / 史诗 5 / 传说 8	在面积计价之上叠加的固定项
湿区系数	x1.4	湿区食材整体上浮
肉禽系数	x1.25	可出现在肉摊的食材上浮
大件补贴	>=4 格 x1.15； >=6 格 x1.3	大体积食材抵消占格劣势

3.2 摊型与路线卡片

摊型共 5 类：叶菜摊、根茎摊、蛋豆摊、水产摊与肉摊。不同摊型的默认货堆数量、装箱推进速率与出货范围各不相同，并与食材定义中的 stalls 字段对应。

路线卡片共 13 种类型，各菜场按独立权重表抽取：

卡型	玩法作用
stall / paystall	免费或付费的摊位，进入翻堆界面
freebie / favor	白捡与人情，直接获得食材
empty / deadend	空摊与死路，消耗天色但无收获
fork / branch	分岔，改变后续路线走向
deep	纵深摊位，收益与风险同时抬高
recipe	油纸菜谱
talk	对话事件，弹出事件面板
gather	采集小游戏
gate	关卡门，按条件放行

3.3 菜谱与菜品

菜谱表共 58 条，其中 18 条为预留未上架条目，玩家可见 40 条。每条菜谱由 RecipeDef 描述，包含 id、菜名、描述、分组、稀有度、所需食材声明 needs、匹配函数 match 与出菜函数 cook，以及经验与首次经验字段。

菜谱有三条解锁来源，互不重叠：

- * 开局赠送 3 本：家常小炒、番茄炒蛋、小葱豆腐。
- * 烹饪台升级：0-9 级每升一级赠送 1 本，共 9 本。
- * 厨艺等级：在 2/4/6/8/10/11/12/13/14/15 等 11 个节点解锁。
- * 集市油纸：按菜场划分的油纸池共 18 条，只能在集市捡到，台子与厨艺不赠送。

3.4 资源与成长

资源/维度	取值范围	获取与消耗
金币	无上限	卖食材与熟菜获得；买摊位货、买交通工具、升级家具消耗
体力	0-5	每 30 分钟恢复 1 点；出门消耗 1 点；吃熟菜可恢复
厨艺等级	1-15	仅做菜获得经验；决定菜场、菜谱与家具门槛
冰箱容量	18-72 格	冰箱升级提升，同格同物最多叠 10 份
出门干区	4x3 - 8x6 格	菜篮 0-9 级提升

出门湿区	2x3 - 6x6 格	泡沫箱 0-9 级提升
房屋等级	陋屋/精装屋/雅致屋	厨艺 4 与 8 级 + 380/980 金币
交通工具	4 档	按序购买，售价 0/180/420/860 金币

四、可进入场景一览

本软件共实现 5 个渲染场景与十余个浮层面板。场景由 SceneManager 统一注册与切换，浮层由 OverlayManager 挂在固定层级之上，场景切换时自动关闭全部浮层。

场景/面板	标识	可进入方式与内容
加载场景	loading	启动即进入，展示 Logo、进度条与健康游戏忠告
厨房场景	kitchen	加载完成后进入，全部功能的中枢
出门选点场景	destinations	厨房点门进入，选菜场与交通工具
集市场景	market	选定菜场后进入，含路线、翻堆与采集三种模式
特殊市场场景	specialMarket	选点页点特殊市场卡进入，节奏点击玩法
冰箱面板	FridgePanel	厨房点冰箱
烹饪面板	CookPanel	厨房点烹饪台
图鉴面板	DexPanel	厨房点左侧图鉴图标
升级面板	UpgradePanel	厨房点菜篮、泡沫箱或家具箭头
菜篮面板	BasketPanel	集市内点顶部菜篮计数
结算面板	ResultPanel	一局结束自动弹出
检视卡	ItemInspectCard	点击摊桌、菜篮或冰箱内的单件食材
事件面板	EventPanel	走到对话类卡片时弹出
菜谱解锁弹窗	RecipeUnlockPanel	捡到油纸菜谱时弹出
厨艺升级弹窗	CookLevelUpPanel	做菜后厨艺升级时弹出
出门过渡幕	OutingCurtain	出门与回家切场时播放

五、游戏操作方法

5.1 基本操作

- * 全部操作为单指触摸：点击用于选择与确认，按住拖动用于菜篮内移动食材与列表滚动。
- * 厨房场景可横向拖动平移视角；纵向列表（菜场、菜谱、冰箱、图鉴）支持惯性滚动。

- * 菜篮内拖动食材到目标格放下即入篮，双击或点旋转按钮把占格旋转 90 度。
- * 右上角静音按钮切换全局音频开关，该设置只存本地不上云。
- * 各浮层面板点击遮罩空白处或关闭按钮退出。

5.2 天色与撤离规则

- * 每走一张卡消耗一点天色，各菜场天色预算为 10-15 步。
- * 天色未尽时点击回家按钮为从容收工，菜篮内食材完整带回。
- * 天色走完自动判定天黑收摊，可能损失新鲜度或掉落部分物品。
- * 停留在摊位期间摊主装箱进度持续推进，满进度即清场，桌上未入篮食材作废。
- * 死路卡消耗一步且无收获，是路线选择的主要风险。

5.3 菜篮放置规则

- * 湿区食材只能放入左侧泡沫箱区、广告解锁通用行，或菜篮 3 级以上的干区保温底行。
- * 干区食材不能放入湿格，避免浸湿。
- * 活物类食材必须贴底放置。
- * 蛋类等易碎食材与坚硬大件不可直接相邻。
- * 豆腐等易压食材上方不可压放重物。
- * 放置冲突时可选择挤走已有食材，系统随后按扫描顺序自动重排。

5.4 经济与成长规则

- * 金币只有两个来源：卖食材与卖熟菜；熟菜售价在出锅时固定写入。
- * 厨艺经验只有一个来源：做菜。卖货、翻堆与事件都不给经验。
- * 菜场按厨艺 1/3/5/7/9/11/12/13/14 逐个开放，同时要求拥有能到达的交通工具。
- * 家具升级费用按 1.38 的等级次方递增，大件家具还受房屋等级限制。
- * 冰箱满时结算面板强制要求先卖出腾位，不会静默丢弃物品。

5.5 每日限量规则

- * 神捡每日限一次，按本地零点换日，通过存档中的 dailyGodPickDate 字段记录。
- * 每个特殊市场每日限进 2 次，通过 specialVisits 字段按日期与计数记录。
- * 菜篮顶部通用行为广告解锁，仅当次出门有效，回家即失效。
- * 体力每 30 分钟恢复 1 点，上限 5 点，按 staminaAt 时间戳离线结算。

六、软件总体设计

6.1 软件需求概括

本软件采用分层、事件驱动的设计方式。客户端负责游戏循环、场景渲染、触摸交互、玩法规则运算、本地存档与资源加载；后端负责平台登录、JWT 签发、云端存档读写、并发版本校验与跨平台账

号隔离。

软件核心需求包括：

- * 提供稳定的收摊捡漏单局玩法，包括路线生成、翻堆抽货、品质判定、装箱校验与撤离结算。
- * 提供完整的局外成长循环，包括冰箱管理、烹饪出菜、厨艺升级、家具与房屋升级、交通工具购买。
- * 提供多界面 UI，包括五个场景与十余个浮层面板，且在不同屏幕比例下正确布局。
- * 提供可靠的本地存档与云存档，保证退出、断网、重进及跨设备的存档一致性。
- * 提供资源分包与 CDN 缓存机制，控制主包体积并提升启动体验。
- * 玩法规则与渲染解耦，规则层可独立运行数值审计脚本以校验平衡性。

6.2 总体架构设计

软件整体划分为小游戏容器层、核心服务层、玩法模拟层、管理器层、UI 表现层、资源层与后端服务层。层与层之间通过明确接口协作，降低平台 API、渲染对象与业务规则之间的耦合。

```
minigame/game.js      小游戏入口，分包加载与启动兜底
+-- minigame/runtime.js  运行环境识别（微信 / 抖音）
+-- minigame/pixi-adapter/*  DOM / canvas / 触摸事件垫片
+-- minigame/game-bundle.js  Vite 打包产物，入口 src/main.ts
  +-- core/Game.ts      Pixi 应用、设计分辨率与安全区
  +-- core/SceneManager.ts  场景注册与切换
  +-- core/OverlayManager.ts  浮层面板栈
  +-- core/EventBus.ts   全局事件总线
  +-- core/PlatformService.ts  微信 / 抖音 / H5 平台能力适配
  +-- core/PersistService.ts  本地持久化与云同步白名单
  +-- core/BackendService.ts  HTTP 登录、pull、push 封装
  +-- core/CdnAssetService.ts  资源清单、CDN 下载与本地文件缓存
  +-- core/AudioManager.ts   BGM 与音效
  +-- sim/*              纯函数玩法模拟层（食材、菜篮、路线、菜谱等）
  +-- managers/*         RunManager / KitchenManager / CloudSyncManager 等
  +-- scenes/*           五个渲染场景
  +-- gameobjects/*      面板、按钮、地图视图等 UI 对象
cloudfunctions/jiancai-api  CloudBase HTTP 后端服务
```

6.3 模块划分与关系

模块层	代表文件/对象	功能简述
入口层	minigame/game.js、src/main.ts	分包加载、Pixi 初始化、启动同步、场景注册与生命周期绑定
平台层	PlatformService、pixi-adapter	封装 request、storage、login、onHide 与 DOM 垫片
服务层	PersistService、BackendService、CdnAssetService	本地存储、云同步快照、后端 HTTP、资源清单与缓存

模拟层	sim/items、sim/basket、sim/run、sim/recipes 等	纯函数玩法规则，不依赖渲染对象
管理器层	RunManager、KitchenManager、SaveManager、CloudSyncManager	持有运行时状态，串联规则与存档
场景层	LoadingScene、KitchenScene、MarketScene 等	绘制场景、响应触摸、驱动管理器
UI 层	BasketPanel、ResultPanel、CookPanel 等	浮层面板与交互反馈
后端层	cloudfunctions/jiancai-api	平台登录、JWT 鉴权、存档拉取与上传

模块之间遵循“入口调度、服务抽象、模拟层纯函数、管理器持有状态、UI 订阅并触发业务”的协作方式。UI 层不直接操作平台 API 与后端接口，一律经管理器或核心服务完成；玩法规则集中在模拟层，使同一套规则可在不同平台与审计脚本中复用。

```
Scene / Panel -> Manager 方法调用 -> sim 纯函数运算 -> 返回新状态
Manager      -> EventBus.emit(EV.kitchenChanged / runChanged / basketChanged)
Scene / Panel -> EventBus.on(...) -> 局部重绘
SaveManager  -> PersistService.writeJSON(SAVE_KEY, data)
PersistService -> CloudSyncManager.schedule(reason) -> BackendService.push
```

6.4 场景与浮层系统设计

场景通过 SceneManager 注册，按名称切换；切换时先卸载当前场景的显示对象与事件监听，再挂载目标场景，并调用 OverlayManager 关闭全部浮层，避免跨场景残留。浮层容器固定在较高层级，面板对象统一实现关闭与重排两个接口，在屏幕尺寸或安全区变化时可重新布局。

集市场景内部再分三种运行模式：路线模式绘制卡片地图，翻堆模式绘制摊面与货堆，采集模式绘制小游戏场景。三种模式共用同一套顶部信息条与浮层，切换时只重建中部内容。

6.5 主循环与资源加载设计

主循环由 Pixi Ticker 驱动，每帧依次更新当前场景、补间动画与输入状态。加载场景在主循环中平滑插值推进进度条，避免进度跳变。

资源分三级供给：主包内 boot 目录只放启动必需的三张图；subpkg_images 与 subpkg_kitchen 为微信分包，在 game.js 中按序加载，单包超时 3000 毫秒即放行；大图与音频优先从 CloudBase CDN 拉取，按清单版本落地为本地文件缓存，命中缓存后不再走网络，下载失败则回退到分包内的同名资源。

七、核心模块详细设计

7.1 游戏入口与平台适配模块

入口分两段。minigame/game.js 是微信小游戏容器入口，负责识别运行环境、挂载 DOM 与 canvas 垫

片、按序加载图片与厨房两个分包，再 `require` 打包产物；同时内置启动诊断：若 12000 毫秒后仍未产生首帧渲染标记，输出诊断信息便于定位真机问题。开发者工具环境下跳过分包加载，直接引用分包入口。

`src/main.ts` 是游戏逻辑入口，按固定时序完成：Pixi 初始化、音频初始化、注册并切入加载场景、云同步预热与启动拉取、本地存档读取、CDN 清单获取、首屏纹理与音效预加载、注册其余四个场景并切入厨房，最后绑定切后台回调。

`PlatformService` 把平台差异收敛为统一接口：`request`、`getStorage/setStorage`、`login`、`showToast`、`onHide`、`restartMiniProgram` 等。`pixiUnsafeEvalPatch` 在导入阶段就替换 `PixiJS` 的动态代码生成路径，禁用 `eval` 以满足小游戏运行环境限制，并注入自定义 `ADAPTER` 与真机纹理修复。

7.2 场景管理与厨房主场景模块

`KitchenScene` 是最复杂的常驻场景。它按房屋等级与家具等级从布局表读取坐标，拼装房屋底图、四件家具贴图与门，构建可横向平移的宽场景视口，并注册门、菜篮、泡沫箱、冰箱、烹饪台五个热区。

场景订阅 `kitchenChanged`、`cookLeveled`、`recipeUnlocked` 三个事件：存档变更时刷新顶部信息条与家具贴图；厨艺升级与菜谱解锁事件进入弹窗队列，按顺序逐个弹出，避免多弹窗叠加。做菜成功后在烹饪台上方播放经验飘字。

7.3 集市地图与卡片路线模块

路线由 `buildMarketMap` 依据菜场规划表与整数种子生成。规划表为每个菜场声明步数预算、摊位层数、每层宽度区间、死路上限与是否允许纵深摊。生成过程先铺设保底摊位层，再用该菜场的卡片权重表填充其余层，最后按独立概率插入油纸菜谱层，并给每个节点挂上遭遇数据。

`MapView` 负责把节点数据渲染为三车道卡片布局，处理明牌与暗牌两种外观、连线绘制、当前位置标记与选中放大动画。玩家点击候选卡后由 `RunManager` 校验是否可进入，扣减天色，按节点遭遇类型分派到翻堆、对话、白捡、采集或分岔处理。

7.4 翻堆抽货与品质模块

每个摊位节点在建局时预生成一个货堆列表，元素记录食材 `id`、初始品质、是否已揭示、是否已检视、是否已抽出等标记。品质初判仅区分坏货与普通货，坏货概率按是否付费买堆取 6% 或 18%；检视时再结合稀有度与随机数细分为新鲜、上品等档位并写入新鲜度。

神捡注入在建局阶段完成：满足每日限量条件时以 85% 概率把水产摊货堆中的一条普通小鱼替换为神捡专属食材，其显示名在未检视时仍为“小鱼”，检视后才揭示为“神捡.野生大黄鱼”并按专属价格计价。

摊主装箱进度按摊型速率逐帧推进，进度上限 100。进度满时强制离开摊位，桌上未入篮的食材全部作废，构成停留时长的成本约束。

7.5 菜篮装箱模块

菜篮状态由 `BasketState` 描述，包含总列数、总行数、湿区列行数、干区行数、是否有保温底、通用

行是否解锁与已放置物品数组。格位类型由 `basketCellKind` 计算，返回湿格、干格、通用格或不可用四种结果。

放置流程如下：

- * `footprint` 按旋转标记换算实际占格宽高。
- * `canPlace` 依次校验越界、重叠、干湿分区、活物贴底、易碎与易压相邻规则。
- * `tryAutoPlace` 按预排序的起点序列扫描空位，湿货优先尝试湿格，两种旋转都试。
- * `tryDrop` 处理玩家手动拖放；目标位冲突时可挤走已有物品，随后对被挤出的物品重新自动排布。

因为占格来自食材真实体积，装箱本身构成本作的核心决策：玩家需要在高价大件与多件小货之间权衡，并为湿区与干区分别留出形状。

7.6 结算与冰箱模块

撤离由 `RunManager.extract` 产出结算结果，区分从容与被赶两种类型，给出带回物品列表与损失数量。`KitchenManager.receiveExtract` 接收结果：冰箱空位充足时直接入库；不足时把物品挂入待处理队列并标记需要玩家腾位，由结算面板引导勾选卖出，确认后统一提交入库。

冰箱条目由 `FridgeItem` 描述，区分食材与熟菜两类，同类同 `id` 合并到同一格并累加份数，单格上限 10 份。食材单价按 `id`、品质、是否检视与新鲜度实时计算；熟菜单价在出锅时固定写入 `value` 字段。

7.7 烹饪与成长模块

每条菜谱声明所需食材类别与份数。下锅前先用匹配函数在冰箱库存中挑出可用食材，不足则在面板上标出缺口。下锅时扣除食材、按出菜函数生成熟菜条目、写入售价与份数，并按稀有度基础经验、份数追加经验与稀有度系数结算厨艺经验。

经验累加后与升级表比对，跨级时抬升等级并把溢出经验带入下一级，满级后经验条停在满值。升级触发 `cookLeveled` 事件，由厨艺升级弹窗集中展示新开放的菜场与菜谱。

```
XP_BASE      = { common: 3, rare: 8, epic: 16, legendary: 28 }
XP_PER_PORTION = { common: 1.5, rare: 2.5, epic: 4, legendary: 5.5 }
COOK_MUL      = { common: 1.6, rare: 1.9, epic: 2.2, legendary: 2.6 }
COOK_XP_TO_NEXT = [0, 60, 110, 180, 250, 340, 450, 580,
                    720, 820, 940, 1060, 1180, 1300, 1420]
upgradeCost(id, lv) = round(base[id] * 1.38^lv * earlyDiscount)
```

7.8 特殊市场模块

特殊市场由静态定义表描述：`id`、名称、对应交通工具、每日限次、轮数、底图、风味类型、时机提示文案、背景音乐，以及命中货池与安慰货池的加权列表。场景内实现一个六态阶段机：静默、预告、命中窗口、过迟、结算与结束，各阶段时长由统一的时机常量表控制，命中窗口固定 0.4 秒。

每轮结束按命中评级从对应货池加权抽取一件食材加入本局收获；五轮走完把收获列表交给通用结算面板，与常规收摊共用同一套入库与卖出流程。

7.9 图鉴模块

图鉴不额外存储条目内容，只在存档中保留两个 id 数组：dexSeen 记录见过的食材，dexInspected 记录已完成检视的食材；recipesCooked 记录已做成的菜品。面板按食材分类与菜品分组把定义表分页展示，并用这三个数组决定每个格位显示彩色条目、灰度剪影还是完全空白。

7.10 存档与云同步模块

本地存档以单个 JSON 写入平台存储，键名为 jiancai_save。SaveManager 在读档时统一做字段规范化与旧字段迁移，并按 staminaAt 时间戳补算离线体力恢复。写入采用防抖，切后台时立即强制落盘。PersistService 维护云同步白名单，只有白名单内的键会被打包上云；Token、匿名设备 id 与音量设置明确排除在外。CloudSyncManager 负责启动拉取与增量上行：

- * 启动时以云端为权威，云端 updatedAt 更新则导入并通知重载。
- * 上行请求防抖 1500 毫秒，失败按指数退避重试，连续失败 5 次后转入 60 秒低频重试。
- * 推送体携带 schemaVersion、updatedAt、baseRemoteUpdatedAt 与客户端指纹，供服务端做乐观并发校验。
- * 服务端返回 409 STALE_UPDATE 时，用响应体中的服务端快照覆盖本地，保证多端一致。

7.11 音频与 CDN 资源模块

AudioManager 统一管理背景音乐与音效：厨房、出门与各菜场各有对应曲目，音效在启动阶段预加载。音量与静音设置只存本地，不进入云同步。

CdnAssetService 启动时拉取资源清单，按前缀规则判断某个资源路径应走 CDN 还是包内；CDN 资源下载后写入以版本号命名的本地缓存目录，后续启动直接命中缓存文件，避免重复下载。

八、数据结构设计

8.1 食材定义 ItemDef

```
type Zone = "dry" | "wet";
type Quality = "rotten" | "common" | "fresh" | "premium" | "god";
type StallId = "leaf" | "root" | "egg" | "fish" | "meat";

interface ItemDef {
  id: string; // 唯一标识
  name: string; // 中文名
  w: number; h: number; // 菜篮占格宽高
  zone: Zone; // 干湿分区
  rarity: Rarity; // common | rare | epic | legendary
  stalls: StallId[]; // 可出现的摊型
  color: number; // 主色，用于占位与描边
  vegetable?: boolean; fragile?: boolean; live?: boolean;
  bulky?: boolean; hard?: boolean; squeezable?: boolean;
  blurb: string; blurbRotten?: string;
  prices: { common: number; fresh: number; premium: number; god?: number };
}
```

8.2 菜篮状态 BasketState

```
interface BasketItem {
    uid: string; defId: string; quality: Quality;
    inspected: boolean; freshness: number;
    x: number; y: number; rot: 0 | 1;
    pinned: boolean; dampened: boolean; broken?: boolean;
}

interface BasketState {
    cols: number; rows: number;
    wetCols: number; wetRows: number; dryRows: number;
    insulatedBottom: boolean; // 菜篮 >=3 级, 干区底行可当湿格
    flexUnlocked: boolean; // 顶部广告通用行, 仅本局有效
    items: BasketItem[];
}

BAG_DRY_COLS = [4, 4, 5, 5, 5, 6, 6, 7, 8]
BAG_ROWS = [3, 4, 4, 4, 5, 5, 5, 6, 6]
FOAM_WET_COLS = [2, 2, 3, 3, 4, 4, 5, 5, 6, 6]
FOAM_WET_ROWS = [3, 4, 4, 4, 4, 5, 5, 5, 6]
```

8.3 单局运行状态 RunState

```
interface RunState {
    seed: number; // 本局随机种子, 可复现
    marketId: MarketId; // 所在菜场
    mode: "map" | "rummage" | "play";
    map: MarketMap; // 卡片路线
    stepsMax: number; stepsLeft: number; // 天色预算与剩余
    atNodeId: string; options: string[]; // 当前位置与候选卡
    piles: Record<string, PileEntry[]>; // 各摊位货堆
    packing: Record<string, number>; // 各摊位装箱进度
    freePass: number; slowNodes: string[]; // 事件产生的临时修正
    bag: ExtractedItem[]; flags: string[];
    returnStack: string[]; // 支线场景返回栈
    ended: boolean; extract?: ExtractResult; lastEvent?: EventView;
}

interface ExtractResult {
    kind: "safe" | "messy"; items: ExtractedItem[];
    lost: number; needsPick?: boolean;
}
```

8.4 本地存档结构 KitchenSave

```
interface KitchenSave {
    version: 1;
```

```
money: number;           // 金币
stamina: number; staminaAt: number; // 体力与上次结算时间戳
fridge: FridgeItem[];     // 冰箱条目
furnLevels: { fridge; table; basket; foam }; // 四件家具 0-9 级
houseLevel: number;       // 0 陋屋 / 1 精装屋 / 2 雅致屋
dexSeen: string[]; dexInspected: string[]; // 食材图鉴
recipesCooked: RecipeId[]; recipesFound: RecipeId[];
dailyGodPickDate: string; // 神捡每日限量
lastSeenAt: number;
level: number; xp: number; // 厨艺等级 1-15 与本级经验
seenCards: string[];      // 明牌记忆, 存 "菜场:卡型"
vehicle: VehicleId; vehicles: VehicleId[];
specialVisits: Record<string, { date: string; count: number }>;
basketLevel: number; fridgeExtra: boolean; // 旧字段, 仅用于迁移
}
```

8.5 云端存档文档结构

云端按平台分集合存放：微信与匿名用户写入 jiancai_playerData，抖音用户写入 jiancai_tt_playerData，实现账号隔离。单用户单文档。

字段	类型	说明
userId	string	平台 openid 派生的用户标识，唯一定位文档
platform	string	wx / dy / tap / anon
schemaVersion	number	存档结构版本，当前为 1
updatedAt	number	客户端产生的更新时间戳，用于乐观并发校验
baseRemoteUpdatedAt	number	本次推送所基于的云端版本时间戳
clientFingerprint	string	客户端指纹，最长 200 字符
payload	object	键为存档 key，值必须为字符串，总大小上限 256KB
payloadKeys	string[]	payload 的键列表，便于服务端审计
lastWriteAt	number	服务端实际写入时间

8.6 云同步白名单

本地存储键	是否上云	说明
jiancai_save	是	唯一上云的核心存档
jiancai_token	否	后端 JWT 缓存，仅本地
jiancai_anon_id	否	匿名设备标识，仅本地
jiancai_audio	否	音量与静音设置，仅本地

jiancai_cloud_meta	否	云同步元信息，仅本地
--------------------	---	------------

九、数据接口设计

9.1 本地存储接口

```
PersistService.readJSON<T>(key: string, fallback: T): T
PersistService.writeJSON(key: string, value: unknown): void
PersistService.subscribeCloudImport(cb: (info: CloudImportInfo) => void)
PersistService.exportSnapshot(): Record<string, string> // 仅白名单内的键
PersistService.importSnapshot(payload, reason): string[] // 返回被改写的键

SaveManager.load(): KitchenSave // 读档 + 字段迁移 + 离线体力结算
SaveManager.flush(): void // 立即落盘
```

9.2 后端 HTTP 接口

后端部署为 CloudBase 云函数并通过 HTTP 访问服务暴露，挂载前缀为 /jiancai-api，全部业务接口为 POST JSON，请求超时 10 秒。

接口	鉴权	功能与返回
GET POST /health	无	健康检查，返回 ok 与服务端时间戳
POST /login	无	wx/dy code2session 或匿名登录，签发 JWT，默认有效期 7 天
POST /save/pull	Bearer	拉取当前用户存档，返回 exists、schemaVersion、updatedAt、payload
POST /save/push	Bearer	上传存档，Upsert 单文档，做大小与版本校验，返回写入模式与字节数

推送接口的校验链与错误码：

状态码	错误码	触发条件
400	BAD_SCHEMA	schemaVersion 缺失或非正数
400	BAD_UPDATED_AT	updatedAt 缺失或非正数
400	BAD_BASE_REMOTE_UPDATED_AT	baseRemoteUpdatedAt 非法
400	BAD_PAYLOAD	payload 不是对象
400	BAD_PAYLOAD_VALUE	payload 中存在非字符串值
409	STALE_UPDATE	云端已有更新版本，响应体携带服务端快照
413	PAYLOAD_TOO_LARGE	payload 序列化后超过 256KB 上限

404	NOT_FOUND	请求路径未注册
500	INTERNAL	未预期异常，服务端记录完整堆栈

9.3 平台能力接口

```
Platform.request(opts): Promise<Response>    // 微信 wx.request / H5 fetch
Platform.getStorage(key) / setStorage(key, v) // 平台本地存储
Platform.login(): Promise<{ code: string }>  // 平台登录票据
Platform.showToast(text, icon)
Platform.onHide(cb)                          // 切后台，触发落盘与云端 flush
Platform.restartMiniProgram(): boolean      // 云端存档覆盖本地后重启
```

9.4 CDN 资源接口

```
CdnAssetService.fetchManifest(): Promise<void> // 拉取资源清单
CdnAssetService.resolve(path): string          // 包内路径 -> 实际可用路径
CdnAssetService.isCdnPath(path): boolean       // 按前缀判断走 CDN 还是包内
preloadTextures(paths, onProgress): Promise<void>
```

十、出错处理设计

10.1 网络异常处理

- * 全部后端请求设置 10 秒超时，超时按失败处理，不阻塞游戏进行。
- * 启动云同步拉取设置 2500 毫秒上限，超时直接以本地存档进入游戏，后台继续重试。
- * 上行失败按指数退避重试，退避上限 30 秒；连续失败 5 次后转入 60 秒低频重试，避免频繁打点。
- * Token 失效时自动重新登录换取新 Token 并重放当次请求。
- * 断网状态下全部玩法仍可正常进行，存档写在本地，恢复网络后自动补传。

10.2 数据冲突与存档异常处理

- * 推送采用乐观并发：携带基线版本时间戳，服务端发现云端更新则返回 409 并附带服务端快照。
- * 客户端收到 409 后用服务端快照覆盖本地，提示玩家已恢复云端存档并重启小游戏，保证多端唯一真源。
- * 读档时统一走字段规范化：缺失字段补默认值，非法类型丢弃，旧字段迁移到新结构，避免版本升级后崩溃。
- * 存档解析失败时回退到全新存档而非抛出异常，同时打印原始内容便于定位。
- * 推送前校验 payload 大小与值类型，超限或非法直接拒绝，防止写入损坏文档。

10.3 资源与运行时异常处理

- * 分包加载单包超时 3000 毫秒即放行，缺图时以占位色块渲染，不中断启动。

- * CDN 资源下载失败时回退到分包内同名资源；两者都缺失时按占位处理并输出资源路径日志。
- * 首屏预加载整体设 12000 毫秒上限，到时无论是否加载完成都进入厨房，避免卡在加载页。
- * 启动 12000 毫秒后仍未产生首帧渲染标记时输出诊断信息，便于定位真机渲染环境问题。
- * WebGL 初始化失败时逐级降级到 Canvas 渲染，保证低端机可运行。
- * PixiJS 的动态代码生成路径在导入阶段即被替换，规避小游戏环境禁用 eval 造成的运行时崩溃。

十一、性能优化设计

11.1 包体与资源优化

- * 主包内只保留加载页所需的三张图，其余图片与音频拆入 subpkg_images 与 subpkg_kitchen 两个分包。
- * 大图与音频优先由 CloudBase CDN 分发，按清单版本落地本地文件缓存，二次启动不再走网络。
- * 首屏只预加载厨房当前房屋与家具等级实际用到的贴图，其余贴图按进入场景时懒加载。
- * 构建产物为单文件 IIFE bundle，减少小游戏环境的模块解析开销。

11.2 渲染与交互优化

- * 场景切换时销毁上一场景的显示对象与事件监听，避免纹理与监听器泄漏。
- * 顶部信息条、菜篮网格等高频刷新区域只重绘变化部分，不整屏重建。
- * 纵向长列表按可视区裁剪，滚动时复用行对象。
- * 补间动画集中由 TweenManager 驱动，统一在主循环内更新，避免各处自建定时器。
- * 事件总线按需订阅与退订，面板关闭即解绑，防止后台面板参与重绘。

11.3 存档与后端负载优化

- * 本地写档防抖合并高频变更，切后台时才强制立即落盘。
- * 云同步只上传白名单内的核心存档键，payload 为字符串映射，控制单次传输量。
- * 上行请求防抖 1500 毫秒，把一段时间内的多次变更合并为一次推送。
- * 服务端按 userId 单文档读写，pull 与 push 各只有一次数据库操作，适合小游戏高频轻量同步。
- * payload 大小硬上限 256KB，在服务端拒绝超限写入，保护数据库与函数执行时间。

11.4 玩法层可测性优化

src/sim 目录为纯函数模拟层，不引用任何渲染对象与平台 API，同一份规则既供游戏运行，也供数值审计脚本 scripts/audit-balance.ts 直接调用，用于批量校验食材定价、菜谱经验与解锁节奏是否符合预期，降低数值改动引入回归问题的风险。

十二、结论

深圳幸运呱科技有限公司扫荡菜场小游戏软件 V1.0.0 围绕“收摊捡漏”这一原创玩法情境，完整实现

了集市卡片路线、摊面翻堆与检视、占格菜篮装箱、收摊结算、冰箱与烹饪、厨艺成长与内容解锁、图鉴收集、特殊市场节奏玩法，以及本地存档与云端同步等全部功能模块。

软件客户端采用 TypeScript 与 PixiJS 实现，玩法规则以纯函数模拟层与渲染层分离；资源经分包与云端 CDN 分发并做本地缓存；后端基于腾讯云 CloudBase 云函数与 HTTP 访问服务实现平台登录、JWT 鉴权、存档读写与乐观并发校验。整体具备清晰的分层结构、明确的数据结构与接口边界、完备的异常处理与性能优化措施，满足软件著作权登记文档鉴别材料对技术说明的要求。